



2025-2 데이터문제해결및실습1

10주차-2

## [경진대회2] 범주형 데이터 이진분류\_2 & 실습 및 과제 1

---

세종대학교

인공지능데이터사이언스학과

박동현 교수



- 본 강의는  
골든래빗  
출판사의  
머신러닝/딥  
러닝  
문제해결전략  
이 제공하는  
강의 교안에  
기반함.

★★★ 반드시 내 것으로 ★★★

#MUSTHAVE

탄탄한 기본기 + 전략적 사고로 문제해결 역량을 레벨업하자

# 머신러닝 · 딥러닝 문제해결 전략



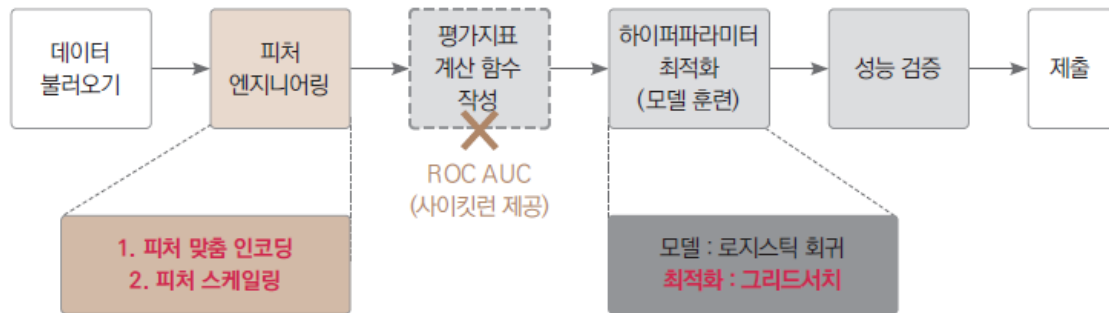
Chapter

07

[경진대회]  
범주형 데이터  
이진분류\_2



## 7.4 성능 개선 I



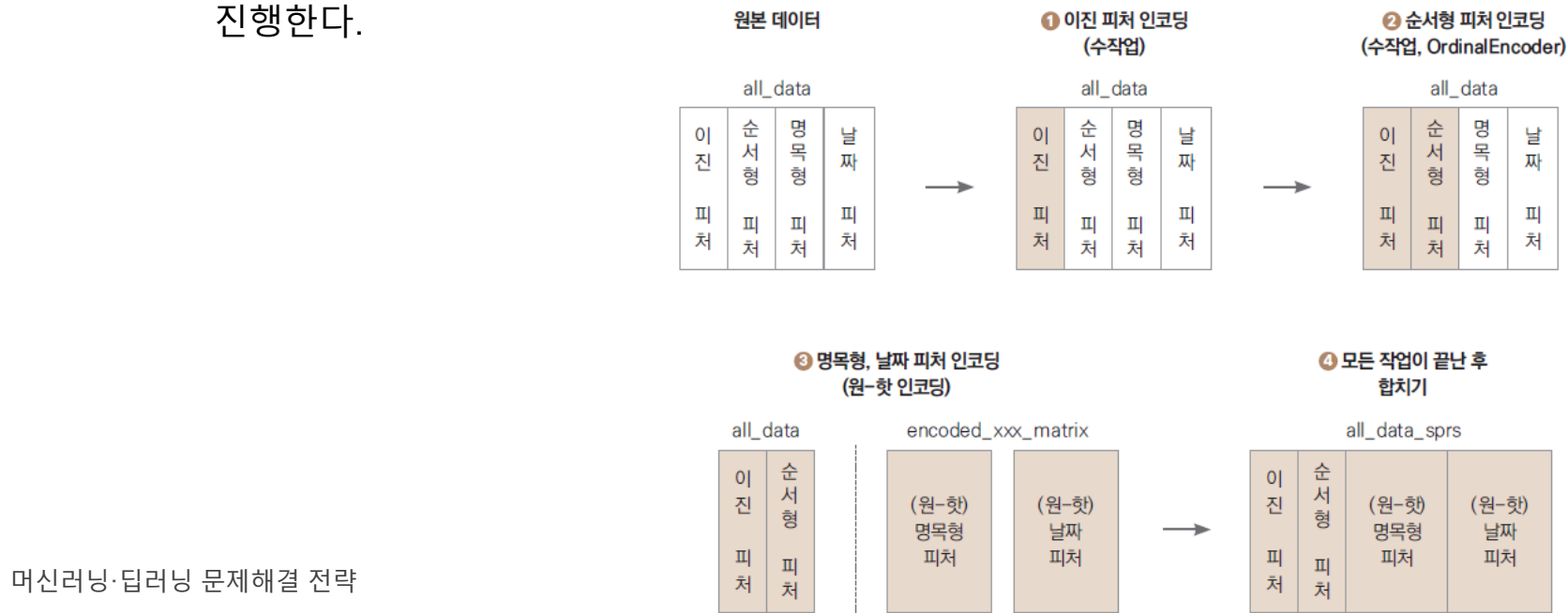
성능 향상을 위해 다음 세 가지에 주안점을 두어 모델링을 진행한다.

1. 피처 맞춤 인코딩
2. 피처 스케일링
3. 하이퍼파라미터 최적화

## 7.3 베이스라인 모델

### 7.4.1 피처 엔지니어링 I : 피처 맞춤 인코딩

- 베이스라인에서는 모든 피처를 일괄적으로 원-핫 인코딩했지만, 피처 특성에 맞게 인코딩한다면 성능이 더 좋아질 수 있다. 이진 피처, 순서형 피처, 명목형 피처, 날짜 피처 순으로 인코딩을 진행한다.



## 7.3 베이스라인 모델

### 7.4.1 피처 엔지니어링 I : 피처 맞춤 인코딩

#### 데이터 합치기

- 인코딩 전에 훈련 데이터와 테스트 데이터를 합쳐 `all_data`를 만들고 타깃값은 제거한다.

#### 이진 피처 인코딩

```
all_data['bin_3'] = all_data['bin_3'].map({'F':0, 'T':1})  
all_data['bin_4'] = all_data['bin_4'].map({'N':0, 'Y':1})
```

#### 순서형 피처 인코딩 (ord\_1, ord\_2 피처 인코딩)

```
ord1dict = {'Novice':0, 'Contributor':1,  
            'Expert':2, 'Master':3, 'Grandmaster':4}  
ord2dict = {'Freezing':0, 'Cold':1, 'Warm':2,  
            'Hot':3, 'Boiling Hot':4, 'Lava Hot':5}  
  
all_data['ord_1'] = all_data['ord_1'].map(ord1dict)  
all_data['ord_2'] = all_data['ord_2'].map(ord2dict)
```

## 7.3 베이스라인 모델

### 7.4.1 피처 엔지니어링 I : 피처 맞춤 인코딩

#### 순서형 피처 인코딩 (ord\_3, ord\_4, ord\_5 인코딩)

```
from sklearn.preprocessing import OrdinalEncoder

ord_345 = ['ord_3', 'ord_4', 'ord_5']

ord_encoder = OrdinalEncoder() # OrdinalEncoder 객체 생성
# ordinal 인코딩 적용
all_data[ord_345] = ord_encoder.fit_transform(all_data[ord_345])

# 피처별 인코딩 순서 출력
for feature, categories in zip(ord_345, ord_encoder.categories_):
    print(feature)
    print(categories)
```

## 7.3 베이스라인 모델

### 7.4.1 피처 엔지니어링 I : 피처 맞춤 인코딩

#### 명목형 피처 인코딩

- 명목형 피처는 순서를 무시해도 되기 때문에 원-핫 인코딩을 적용한다.

```
nom_features = ['nom_' + str(i) for i in range(10)] # 명목형 피처
```

이 명목형 피처를 원-핫 인코딩해 별도 행렬에 저장하고, 이어서 `all_data`에서 명목형 피처를 삭제한다.

```
# 정상 실행되는 코드
from sklearn.preprocessing import OneHotEncoder

onehot_encoder = OneHotEncoder() # OneHotEncoder 객체 생성
# 원-핫 인코딩 적용
encoded_nom_matrix = onehot_encoder.fit_transform(all_data[nom_features])

encoded_nom_matrix
```

마지막으로 `all_data`에서 기존 명목형 피처를 삭제한다.

```
all_data = all_data.drop(nom_features, axis=1) # 기존 명목형 피처 삭제
```



## 7.3 베이스라인 모델

### 7.4.1 피처 엔지니어링 I : 피처 맞춤 인코딩

#### 날짜 피처 인코딩

- day와 month는 날짜 피처다. 7.2.2절에서 설명했듯이 날짜 피처에도 원-핫 인코딩을 적용한다.

```
date_features = ['day', 'month'] # 날짜 피처

# 원-핫 인코딩 적용
encoded_date_matrix = onehot_encoder.fit_transform(all_data[date_features])

all_data = all_data.drop(date_features, axis=1) # 기존 날짜 피처 삭제

encoded_date_matrix
```

## 7.3 베이스라인 모델

### 7.4.2 피처 엔지니어링 II : 피처 스케일링

- **피처 스케일링(feature scaling)**: 서로 다른 피처들의 값의 범위가 일치하도록 조정하는 작업. 피처 스케일링이 필요한 이유는 수치형 피처들의 유효 값 범위가 서로 다르면 훈련이 제대로 안 될 수도 있기 때문이다.

#### 순서형 피처 스케일링

- 다른 피처들과 범위를 맞추기 위해 순서형 피처에 min-max 정규화를 적용한다. min-max 정규화는 피처 값의 범위를 0~1로 조정한다.

```
from sklearn.preprocessing import MinMaxScaler

ord_features = ['ord_' + str(i) for i in range(6)] # 순서형 피처
# min-max 정규화
all_data[ord_features] = MinMaxScaler().fit_transform(all_data[ord_features])
```

## 7.3 베이스라인 모델

### 7.4.2 피처 엔지니어링 II : 피처 스케일링

#### 인코딩 및 스케일링된 피처 합치기

- 현재 all\_data에는 이진 피처와 순서형 피처가 인코딩돼 있다. 명목형 피처와 날짜 피처는 원-핫 인코딩되어 각각 encoded\_nom\_matrix와 encoded\_date\_matrix에 저장돼 있다. 세 데이터를 합친다.
- all\_data를 CSR 형식으로 만들어 합친다. 사이파이가 제공하는 csr\_matrix()는 전달받은 데이터를 CSR 형식으로 바꿔준다.

```
from scipy import sparse

# 인코딩 및 스케일링된 피처 합치기
all_data_sprs = sparse.hstack([sparse.csr_matrix(all_data),
                               encoded_nom_matrix,
                               encoded_date_matrix],
                               format='csr')
```

- all\_data\_sprs

## 7.3 베이스라인 모델

### 7.4.2 피처 엔지니어링 II : 피처 스케일링

#### 데이터 나누기

- 훈련 데이터와 테스트 데이터를 나눈다.

```
num_train = len(train) # 훈련 데이터 개수

# 훈련 데이터와 테스트 데이터 나누기
X_train = all_data_sprs[:num_train] # 0~num_train - 1행
X_test = all_data_sprs[num_train:] # num_train~마지막 행

y = train['target']
```

- 훈련 데이터를 다시 훈련 데이터와 검증 데이터로 나눈다.

```
from sklearn.model_selection import train_test_split

# 훈련 데이터, 검증 데이터 분리
X_train, X_valid, y_train, y_valid = train_test_split(X_train, y,
                                                    test_size=0.1,
                                                    stratify=y,
                                                    random_state=10)
```

## 7.3 베이스라인 모델

### 7.4.3 하이퍼파라미터 최적화

- 6장에서 배운 그리드서치를 활용해 로지스틱 회귀 모델의 하이퍼파라미터를 최적화한다.

```
%%time

from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LogisticRegression

# 로지스틱 회귀 모델 생성
logistic_model = LogisticRegression()

# 하이퍼파라미터 값 목록
lr_params = {'C':[0.1, 0.125, 0.2], 'max_iter':[800, 900, 1000],
             'solver':['liblinear'], 'random_state':[42]}

# 그리드서치 객체 생성
gridsearch_logistic_model =
GridSearchCV(estimator=logistic_model,

              param_grid=lr_params,
              scoring='roc_auc', # 평

              cv=5)

# 그리드서치 수행
gridsearch_logistic_model.fit(X_train, y_train)

print('최적 하이퍼파라미터:',
      gridsearch_logistic_model.best_params_)
```

## 7.3 베이스라인 모델

### 7.4.4 모델 성능 검증

- 베이스라인처럼 검증 데이터로 모델 성능을 검증한다. 먼저 검증 데이터로 타깃 예측값을 구한다.

```
y_valid_preds = gridsearch_logistic_model.predict_proba(X_valid)[: , 1]
```

- 이어서 검증 데이터 ROC AUC를 구한다.

```
from sklearn.metrics import roc_auc_score # ROC AUC 점수 계산 함수

# 검증 데이터 ROC AUC
roc_auc = roc_auc_score(y_valid, y_valid_preds)

print(f'검증 데이터 ROC AUC : {roc_auc:.4f}')
```

### 7.4.5 예측 및 결과 제출

- 제출 파일을 만들어 커밋 후 제출한다.

## 7.5 성능 개선 II

훈련 데이터 전체를 사용해 모델을 훈련해본다. `train_test_split()`으로 훈련 데이터(90%)와 검증 데이터(10%)로 나누는 부분과 연관된 코드를 제외하고 나머지 모든 절차는 7.4절과 동일하게 수행한다.

